# 회귀 모델: 선형회귀와 로지스틱회귀

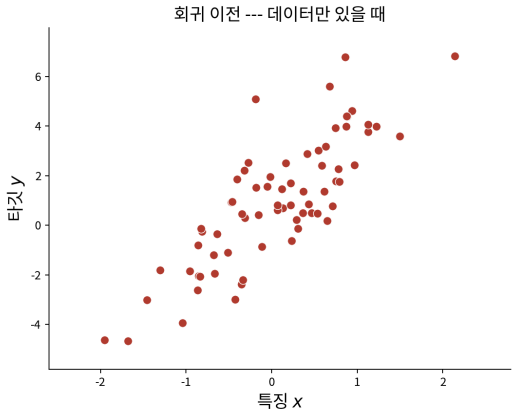
## 선형회귀: 모델, 비용함수, 경사하강법 · 정규방정식과 “같은 모델, 다른 출력”의 다리 · 로지스틱회귀: 시그모이드에서 PR-AUC까지

Machine Learning 1 · 2주차

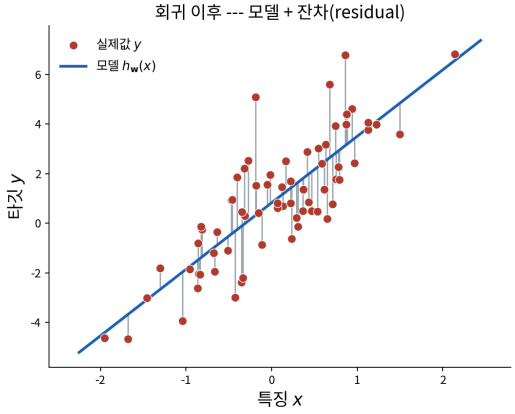
한국과학영재학교 (KSA)

Week 2

# 선형회귀란?



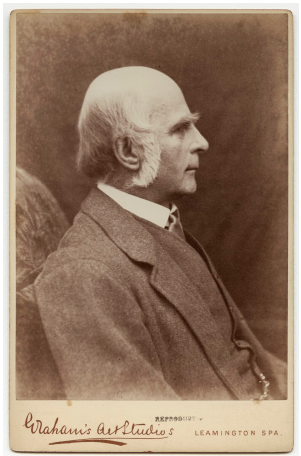
# 선형회귀란?



# 이번 주차 개요

이 주차를 마치면:

- **모델 · 비용함수 · 최적화** 뼈대로 선형·로지스틱회귀 조립
- MSE 그래디언트와 경사하강법, **학습률·조건수**와 수렴
- 정규방정식 $\boldsymbol{w}^* = (X^T X)^{-1} X^T y$ 유도와 **사영** 해석
- 시그모이드 $\rightarrow$ 교차 엔트로피, 혼동행렬 · F1 · PR-AUC 평가




Francis Galton — 1886년 논문 “Regression towards Mediocrity in Hereditary Stature”에서 처음 “회귀(regression)”라는 말을 씀

# 2.1 선형회귀: 모델, 비용함수, 경사하강법

# 첫 완전한 알고리즘: 세 부분 뼈대

알고리즘의 뼈대는 항상 같은 세 부류:

1. **모델** — 입력을 출력으로 매핑하는 함수의 형태
2. **비용함수**(loss) — “얼마나 틀렸는지”를 숫자로 재는 재
3. **최적화 알고리즘** — 파라미터를 그 숫자가 줄어드는 방향으로 바꾸는 방법

# 모델: 특징의 가중합

선형회귀는 입력 $\boldsymbol{x} = (x_1, \ldots, x_n)$ 에서 출력을 이렇게 예측한다:

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n, \qquad h_{\boldsymbol{w}} : \mathbb{R}^n \to \mathbb{R}$$

- $w_0$ = **절편**(bias), $w_1, \ldots, w_n$ = 각 특징의 **가중치**

# 파라미터는 어떻게 정해지는가: “맞춤” 작업

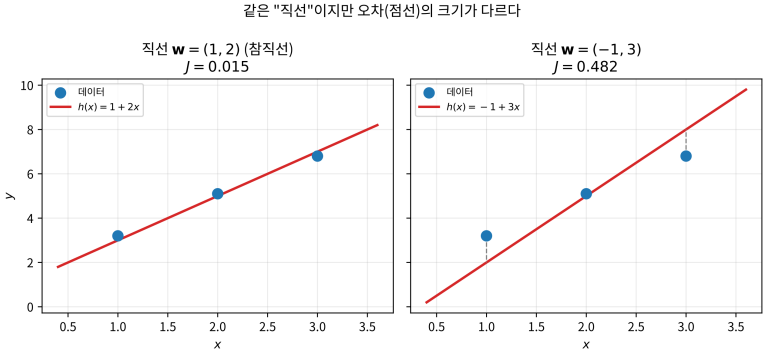
$h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x}$ 에서 우리가 조절할 수 있는 것은 $\boldsymbol{w} = (w_0, \ldots, w_n)$ 뿐이다.

**학습 알고리즘의 임무**

학습 데이터 $(\boldsymbol{x}^{(i)}, y^{(i)})$ 에 대해 예측값 $h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)})$ 가 실제 출력 $y^{(i)}$ 에 가깝게 되는 $\boldsymbol{w}$ 를 찾는다.

특징이 하나뿐이면 $(n = 1)$ $h_{\boldsymbol{w}}(\boldsymbol{x}) = w_0 + w_1 x$ 는 **직선**이다. 임무는 무수히 많은 직선 중 **데이터 점들에 가장 잘 들어맞는 한 줄**을 찾는 **맞춤(fitting)** 작업이 된다.

# “잘 들어맞다”를 숫자로




**비용함수가 알고리즘의 진짜 출발점**

# 비용함수: 평균제곱오차

$$J(\boldsymbol{w}) = \frac{1}{2m} \sum_{i=1}^{m} \left( h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)}) - y^{(i)} \right)^2$$

- **제곱은 두 문제를 동시에 해결**: 항상 양수 + 매끄럽게 미분 가능
- 앞의 $\frac{1}{2}$ 은 미분할 때 지수 2와 약분되라고 붙인 것. **평가 지표**로 쓸 때는 $\frac{1}{2}$ 없이 $\text{MSE} = \frac{1}{m} \sum_i (\hat{y}^{(i)} - y^{(i)})^2$, $\text{RMSE} = \sqrt{\text{MSE}}$ (원래 $y$ 와 같은 단위)

# $\frac{1}{2}$, 그리고 볼록함수

- $J(\boldsymbol{w})$ 는 $\boldsymbol{w}$ 에 대해 **이차함수**(quadratic): $w_j w_k$ (2차), $w_j$ (1차), 상수의 합으로만 이루어짐.

**볼록함수 판정 — 헤시안 한 계산**

이차함수가 볼록함수(convex)인지는 **헤시안**(Hessian, 2차 편미분 행렬)이 양정치인지 한 계산으로 판정:

$$\frac{\partial^2 J}{\partial \boldsymbol{w} \partial \boldsymbol{w}^T} = \frac{1}{m} X^T X$$

— bias 열을 붙인 데이터 행렬 $X$ 의 $X^T X$ 에 비례.

# 장난감 데이터: $J$ 는 엄밀히 볼록

- 이 행렬의 두 고유값 $\approx 0.120, \approx 5.547$ — **모두 양수** $\rightarrow J$ 는 **엄밀히 볼록**(strictly convex).
- **볼록 이차함수는 전역 최솟값이 하나**

# 고유값 비율 46:1 — “늘어짐”의 첫 단서

$$5.547 / 0.120 \approx 46$$

- 기하학적 뜻: $J(\boldsymbol{w})$ 의 골짜기 단면이 두 고유방향으로 **약 46:1로 늘어진 타원**.

**조건수가 이 절 전체의 “느림과 발산”을 정량화하는 숫자다.**

# 경사하강법 (Gradient Descent)

**그래디언트 반대 방향으로 이동**

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}, \qquad \frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left( h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

- $\alpha$ = **학습률(learning rate)** — 한 걸음의 크기

# 업데이트는 왜 “동시”인가

## 틀린 예: 순차 업데이트

```
w0 -= a * grad0
w1 -= a * grad(w0, w1)
```

둘째 줄의 grad가 이미 바뀐 w0로 계산됨 — “이동 전”과 “이동 후”가 섞인다.

## 맞는 예: 동시 업데이트

```
g0, g1 = grad(w0, w1)
w0 -= a * g0
w1 -= a * g1
```

grad를 전부 계산한 다음에야 w[j] -= ...로 반영 — 두 단계 구조.

**순차 업데이트는 경사 보장이 무너진다**

# 그래디언트는 왜 “오차 × 특징” 형태인가

## (2) $\partial J/\partial w_j$ 가 그 합이 되는 이유

각 데이터점 $i$ 는 오차의 제곱을 $J$ 에 기여. 제곱의 미분에 연쇄법칙을 쓰면 2가 나와 $\frac{1}{2}$ 와 상쇄. $h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)})$ 를 $w_j$ 로 미분하면 $x_j^{(i)}$ **하나**가 남는다. 데이터점 $i$ 의 기여:

$$\left(h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)}) - y^{(i)}\right) \cdot x_j^{(i)} \quad \text{— 오차} \times \text{해당 특징의 값}$$

# 경사하강법 구현 (순수 Python)

```python
def gradient_descent(X, y, alpha, epochs):
    m, n = len(X), len(X[0])
    w = [0.0] * (n + 1)  # w[0]=bias
    for _ in range(epochs):
        grad = [0.0] * (n + 1)
        for i in range(m):
            pred = w[0] + sum(w[j+1] * X[i][j] for j in range(n))
            error = pred - y[i]
            grad[0] += error
            for j in range(n):
                grad[j+1] += error * X[i][j]
        for j in range(n + 1):
            w[j] -= alpha * grad[j] / m
    return w
```

# 수렴 경계: 발산하는 $\alpha$ 는 정확히 계산된다

선형회귀는 $J$ 가 이차함수라 **수렴·발산 경계값을 정확히 계산**할 수 있다. 업데이트는 매 스텝 "현재점과 최솟값 $\boldsymbol{w}^*$ 의 오차"에 고정 행렬을 곱하는 선형 연산:

$$\boldsymbol{w}^{(t+1)} - \boldsymbol{w}^* = \left(I - \frac{\alpha}{m} X^T X\right)(\boldsymbol{w}^{(t)} - \boldsymbol{w}^*)$$

장난감 데이터에서 $\frac{1}{m} X^T X$ 의 고유값이 $\approx\ 0.120$ 과 $\approx\ 5.547$ 이므로, 매 스텝 오차가 고유방향별로 $(1 - \alpha \cdot 0.120)$ 배, $(1 - \alpha \cdot 5.547)$ 배. 두 방향 모두 줄려면 $|1 - \alpha \cdot 5.547| < 1$, 즉

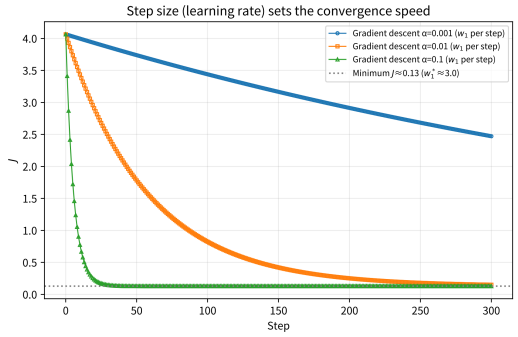
$$0 < \alpha < \frac{2}{5.547} \approx 0.361\ = \alpha^*$$

# $\alpha^* \approx 0.361$ 이 운명을 정한다

| 학습률 $\alpha$ | $\alpha/\alpha^*$ | 운명 |
|---|---|---|
| 0.1 | 0.28배 | 수렴 (조금 느리게) |
| 0.5 | 1.38배 | **발산** — 1.5보다 "조금 작은" 값이지만 |
| 1.5 | 4.2배 | 심각하게 발산 |

$\alpha = 0.5$ **도 발산한다**

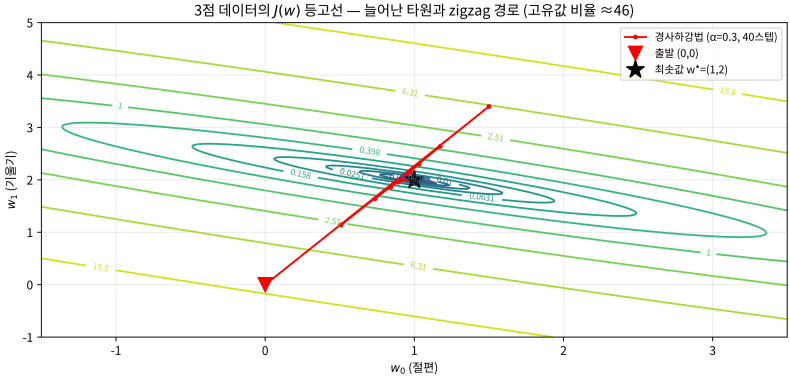
# 1D 단면: 걸음 크기가 속도를 정한다




$w_0 = 1$ 고정, 세 학습률($\alpha = 0.1, 0.01, 0.001$)로 $w_1$만 경사하강 — 스텝별 비용 $J$, 같은 출발점.

- 세 곡선은 **같은 점에서 출발해 같은 최솟값**(점선)으로 수렴.
- $\alpha = 0.1$ 이 가장 빠르게 바닥에 닿고, $\alpha = 0.01$ 은 약 **10배 느리다**.
- $\alpha = 0.001$ 은 300스텝이 지나도 바닥에서 꽤 먼 채로 거의 움직이지 못함.
- 1D 그림은 "속도"를 보여준다 — 학습률은 "가는 방향"이 아니라 **"걸음 수"**를 정하는 하이퍼파라미터.

# 2D 지형: 경로는 왜 일직선이 아닌가




- 등고선이 **늘어진 타원**인 이유 = 앞선 고유값 비율 ($\approx 46$) 이 바로 그 타원의 축 비율이기 때문.
- **타원 등고선에서 대각 방향을 걷어 zigzag**

# 10배 규칙: 실데이터에서는 느림이 흔한 실패모드

| 학습률 | 소음 바닥 도달 스텝 |
|---|---|
| $\alpha = 0.001$ | 3000스텝 내 도달 불가 (최종 $J = 0.154$, 바닥보다 20% 높음) |
| $\alpha = 0.01$ | 489 |
| $\alpha = 0.1$ | 47 |

**소음 바닥** $J_{\text{floor}} = 0.1287$ — 노이즈가 만들어낸, 어떤 직선도 더 낮출 수 없는 이론적 최소 비용.

**학습률 10배 올리면 스텝 수 10배 감소**

# 학습률의 함정: 너무 작은 값

- $\alpha$**가 너무 작으면 수렴이 느리다**

**실전 절차**

"작을수록 안전"이 아니라, **비용 곡선이 매끄럽게 내려가는 가장 큰 학습률을 찾는 것**.
비용 곡선이 거의 수평으로 뻗어 있다면 발산 문제 이전에 "학습률이 너무 작은 것"부터 의심.

**경계는 데이터마다 다르다**

# 손으로 한 번: 세 스텝 직접 계산

아주 작은 데이터셋 $X = [1, 2, 3]$, $y = [3, 5, 7]$ (참값 $y = 2x + 1$), $w_0 = w_1 = 0$ 에서 시작, $\alpha = 0.1$.

초기 비용:

$$J(0,0) = \tfrac{1}{6}\left[3^2 + 5^2 + 7^2\right] = \tfrac{83}{6} \approx 13.83$$

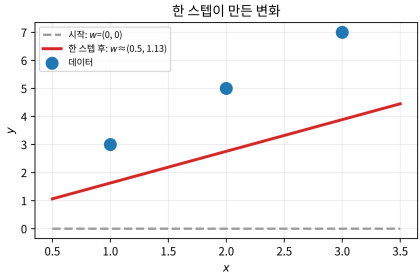
그래디언트:

$$\frac{\partial J}{\partial w_0} = -5, \quad \frac{\partial J}{\partial w_1} \approx -11.33$$

한 스텝 이동:

$w_0 \leftarrow 0 - 0.1(-5) = 0.5$

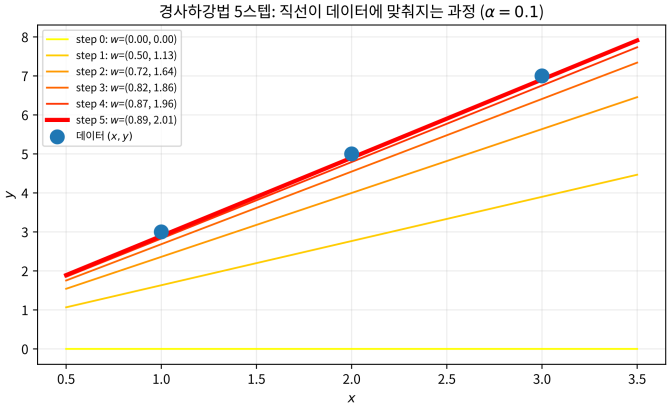
$w_1 \leftarrow 0 - 0.1(-11.33) \approx 1.13$



# 손계산 다섯 스텝의 결과

| 스텝 | $w_0$ | $w_1$ | $J(\boldsymbol{w})$ |
|---|---|---|---|
| 0 (초기) | 0.0000 | 0.0000 | 13.8333 |
| 1 | 0.5000 | 1.1333 | 2.7443 |
| 2 | 0.7233 | 1.6378 | 0.5448 |
| 3 | 0.8234 | 1.8621 | 0.1086 |
| 4 | 0.8687 | 1.9618 | 0.0221 |
| 5 | 0.8894 | 2.0059 | 0.0049 |

$w_1$ 이 참값 2에, $w_0$ 이 참값 1에 매 스텝 빠르게 가까워지고 비용이 **단조 감소**한다.



# 자주 하는 실수: 학습률을 너무 크게 잡기

같은 데이터, 같은 초기값에서 $\alpha = 1.5$ 로만:

```
w0, w1 = 0.0, 0.0
alpha_big = 1.5
for i in range(1, 6):
    g0, g1 = grad(w0, w1, X, y)
    w0 -= alpha_big * g0
    w1 -= alpha_big * g1
    print(f"step {i}: w0={w0:.4f} w1={w1:.4f} cost={cost(w0, w1, X, y):.4f}")
```

```
step 1: w0=7.5000   w1=17.0000   cost=741.13
step 2: w0=-47.2500 w1=-107.5000 cost=39708.03
step 3: w0=353.6250 w1=803.7500  cost=2127480.20
step 4: w0=-2580.5625 w1=-5866.3750 cost=113986311.47
step 5: w0=18896.9062 w1=42956.9375 cost=6107168110.25
```

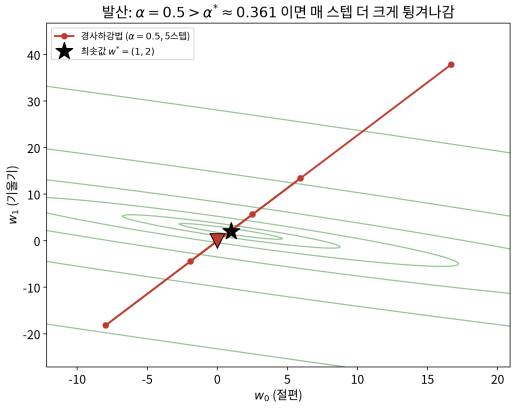
# 발산의 모습과 실전 대응

- **스텝이 너무 커서 발산**

## 실전 대응

손실이 NaN 이나 매우 큰 값으로 튀면, **가장 먼저 의심할 것이 바로 학습률**.
보통 $\alpha = 0.1, 0.01, 0.001$ 처럼 10배씩 줄여가며 손실 곡선이 매끄럽게 내려가는 지점을 찾는다 — 이 "10배씩" 경험칙이 임의가 아니라 데이터 특이적 경계값 $\alpha^*$ 가 존재하기 때문 (앞 슬라이드 확인).

# 그림으로: 발산 궤적




장난감 데이터($X = [1, 2, 3]$)에서 $\alpha = 0.5$ 는 경계값 $\alpha^* \approx 0.361$ 을 넘어 — 5스텝 만에 $w_0, w_1$ 이 부호를 바꿔가며 크기가 계속 커진다.

# 확인 문제: $\alpha = 0.5$ 는?

$\alpha = 0.1$ 은 매끄럽게 줄었고, $\alpha = 1.5$ 는 발산했다. **실행 전에 먼저 예상하라** — $\alpha = 0.5$ 는 수렴하되 0.1보다 빠를지, 여전히 발산할지, 진동하다가 서서히 수렴할지.

| 스텝 | $w_0$ | $w_1$ | $J(\boldsymbol{w})$ |
|---|---|---|---|
| 1 | 2.5000 | 5.6667 | 43.4954 |
| 2 | $-1.9167$ | $-4.3889$ | 136.7638 |
| 3 | 5.9306 | 13.4352 | 430.0336 |
| 4 | $-7.9699$ | $-18.1775$ | 1352.1803 |
| 5 | 16.6925 | 37.8732 | 4251.7440 |

**정답:** $\alpha = 0.5 > \alpha^* \approx 0.361$ 이므로 **발산**. $\boldsymbol{w}$ 가 양수/음수로 **진동**하면서 크기는 계속 커진다.

숫자 0.361 자체가 아니라 **경계값이 존재한다는 것**이 요점: 특징이 $n = 3, 4$ 여도 규칙은 그대로 $0 < \alpha < 2/\lambda_{\max}$ — $\lambda_{\max}$ 는 그냥 그때의 $(n+1) \times (n+1)$ $X^T X/m$ 의 최대 고유값일 뿐, 공식이 바뀌지 않는다.

# 느리게 발산하는 형태가 더 위험

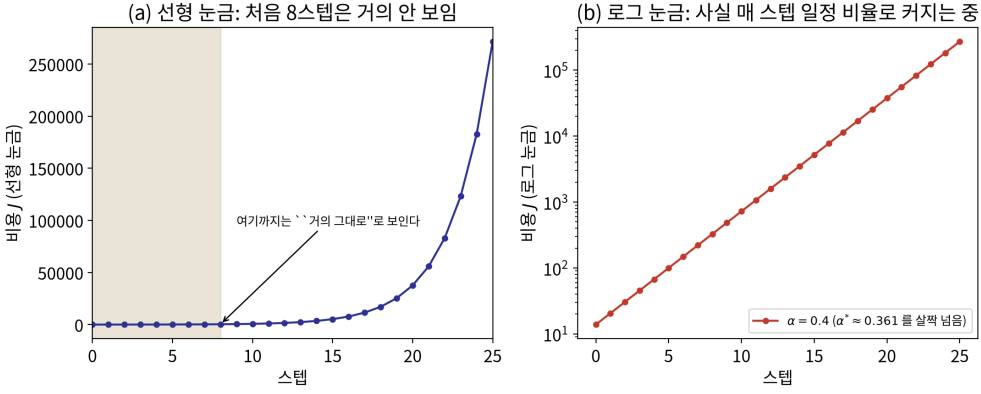
- $\alpha = 1.5$ 처럼 바로 크게 폭발하지 않고 **느리게 발산**하는 형태.

**경계값을 넘은 신호**

비용이 **"줄어든 뒤 다시 올라가기 시작"**하는 순간. 실전에서 이 패턴을 보면 학습률부터 줄여라.

# 그림으로: 느리게 발산하는 비용 곡선

같은 데이터, 같은 경로 -- 눈금만 다르다




$\alpha = 0.4$ 는 경계값 $\alpha^* \approx 0.361$ 을 살짝만 넘겨도 (a) 선형 눈금은 처음 8스텝이 평평해 “안정적”으로 보이지만, (b) 로그 눈금은 첫 스텝부터 같은 비율로 커졌음을 보여준다.

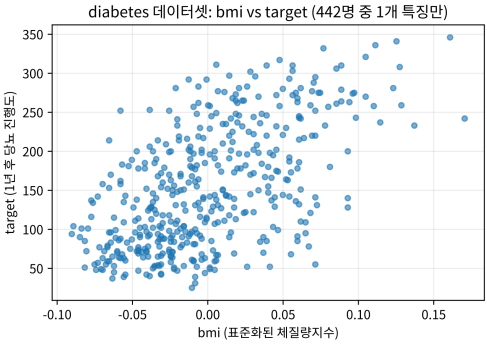
# 실데이터: diabetes 데이터셋 (1/2)

3개 데이터의 장난감을 넘어, **실전 문제** (특징 10개, 442명)에서 경사하강법의 수렴 모양을 확인.

- sklearn `load_diabetes` (당뇨 진행도 예측, 10개 신체 특징) 전체 442명 — Efron, Hastie, Johnstone, Tibshirani (2004) 의 데이터.
- 모든 특징을 **표준화**한 뒤 (2.2절 스케일링의 동기) 두 가지를 재본다.

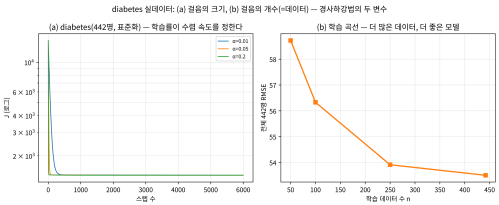
**(1) 세 학습률의 비용 곡선** ($\alpha \in \{0.01, 0.05, 0.2\}$): 표준화 후 $(1/m)X^TX$ 의 최대 고유값 $\lambda_{\max} \approx 4.0$ 이므로 경계 $\alpha^* = 2/\lambda_{\max} \approx 0.50$ — $\alpha = 0.2$ 도 경계 안이라 발산하지 않는다.

**1000스텝 후 $J$: 1439 / 1435 / 1430**




10개 특징 중 하나(bmi)만 골라 본 모습. 출처: `scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset`.

# 실데이터: diabetes 데이터셋 (2/2)




(좌) 세 학습률의 비용 곡선, (우) 데이터 크기 $n$ 에 따른 최종 RMSE의 학습 곡선.

**(2) 학습 곡선** — $n = 50, 100, 250, 442$ 로 학습한 모델을 **전체 442명** 위에서 RMSE로 잴 때:

$$58.7 \to 56.3 \to 53.9 \to 53.5$$

데이터가 작을수록 모델이 **나빠진다** — “배울 것이 적어서”.

수렴 스텝 수는 $n$ 에 대해 단조가 아님 (2869 $\to$ 912 $\to$ 2807 $\to$ 2405) — “데이터가 적으면 빠르다”는 일반 법칙이 아니라, 소음 바닥의 크기 자체가 $n$ 에 따라 달라지기 때문.

# 결과의 핵심 한 줄

**핵심**

전체 442명으로 학습한 경사하강법의 최종 RMSE **53.5** 는, 같은 데이터의 **정규방정식(닫힌 형태) 해의 RMSE와 일치**.
2.2절에서 “경사하강법과 정규방정식은 같은 목적지”라고 할 때의 말이, 장난감 3점을 넘어 **실데이터 위에서 검증**된 셈.

- “닫힌 형태의 답” (2.2절 정규방정식) 과
- “분류” (2.3절 로지스틱회귀) 로 나간다.

# 유명한 사례: 가우스와 사라진 소행성 세레스

- 1801년 발견된 소행성 **세레스**가 **단 41일치** 관측(피아치, 1/1∼2/11) 뒤 태양 뒤로 사라지자, 24세 **가우스**가 그 부정확한 관측만으로 재출현 위치를 정확히 예측.
- 쓴 방법 = **오차 제곱합 최소화** — 지금 이 절의 바로 그 $J(\boldsymbol{w})$.

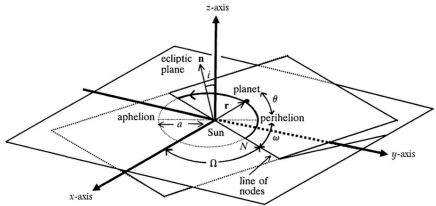


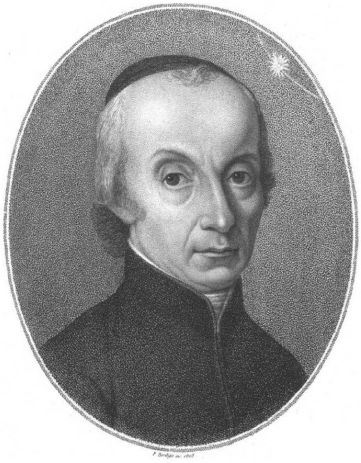

FIGURE 1
Parameters describing the planetary orbit

## 200년 넘게 쓰이는 이유

"아름다워서"가 아니라, 관측(데이터)이 참값에 잡음이 섞인 구조가 **실존 세계에 끊임없이 반복**되기 때문.

**최소제곱법은 직선과 타원 모두에 적용**

궤도를 정하는 6개 숫자 $\boldsymbol{w}$: $a$(장반경), $e$(이심률), $i$(경사각), $\Omega$(승교점 경도), $\omega$(근일점 인수), $\theta$(위치) — 직선의 $(w_0, w_1)$이 6개로 늘어난 것과 본질이 같다.



Giuseppe Piazzi — 1801년 1월 1일
세레스를 처음 관측한 팔레르모 천문대 신부



왜소행성 세레스 — NASA Dawn 탐사선이 2015년
촬영한 실제 컬러 사진

# 자주 묻는 질문 (1)

**Q. 경사하강법이 지역 최솟값에 갇힐 수 있나요?**

- 이 절의 $J(\boldsymbol{w})$ (평균제곱오차)는 $\boldsymbol{w}$ 에 대한 **볼록함수**라 **지역 최솟값이 곧 전역 최솟값** — 어디서 출발하든 결국 같은 최적점에 수렴.
- 지역 최솟값이 실제 골칫거리가 되는 건 **Week 9 신경망처럼 비볼록 손실**을 쓸 때.

**Q. 매 스텝 데이터 전체를 다 봐야 하나요?**

- 위 코드 = 매 스텝 $m$ 개 전부로 그래디언트를 계산하는 **배치 경사하강법**(batch GD).
- 데이터가 수백만 개면 스텝 하나조차 느려지는데, 실전은 작은 묶음(mini-batch)만 보는 **확률적 경사하강법**(SGD)을 훨씬 더 많이 쓴다 — Week 9에서 변형들을 자세히.

# 자주 묻는 질문 (2)

**Q. 학습률은 실전에서 어떻게 정하나요?**

- $J$ 가 이차함수이면 (선형회귀) 이 절의 공식으로 정량적:

$$0 < \alpha < 2/\lambda_{\max}$$

  ($\lambda_{\max} = (1/m)X^T X$ 의 최대 고유값).
- 신경망처럼 이차함수가 아니면 공식이 없으므로, 보통 $\alpha = 0.1 \sim 0.01$ 에서 시작해 비용 곡선을 보고 조절: 매끄럽게 내려가면 더 큰 값으로, 흔들리거나 발산하면 10배 줄인다.
- 파라미터마다 학습률을 자동 조절하는 **적응형 학습률 알고리즘** (SGD 변형, Adam 등)이 실전의 표준 — Week 9.

**Q. 0에서 시작하는 것은 필수인가?**

- 아니요 — 볼록함수는 **어디서 시작하든** 같은 최솟값에 도달 (첫 Q 참고). 0은 관례일 뿐.
- 단, 초기값이 멀수록 스텝 수 ↑ — 특징 표준화 (2.2절)가 실용적인 이유.

# 2.2 정규방정식과 “같은 모델, 다른 출력”이라는 다리

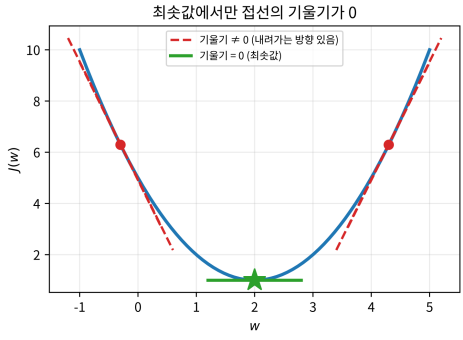
# 정규방정식: 미분으로 한 번에 풀기

$J(\boldsymbol{w})$ 는 $\boldsymbol{w}$ 에 대한 이차함수이므로, 경사하강법 없이 **미분을 0으로 놓아 닫힌 형태(closed-form)로 최적해를 구할** 수도 있다:

$$\nabla_{\boldsymbol{w}} J = \tfrac{1}{m} X^T (X\boldsymbol{w} - y) = 0 \;\Rightarrow\; X^T X \boldsymbol{w} = X^T y$$

$$\Rightarrow \quad \boldsymbol{w}^* = (X^T X)^{-1} X^T y$$

- $X$: 각 행이 $\boldsymbol{x}^{(i)}$ (bias 포함)인 $m \times (n+1)$ 행렬
- $\nabla_{\boldsymbol{w}} J = 0$을 직접 전개해 위 두 줄을 얻는 과정이 이번 장 연습문제(2번)의 핵심.




볼록함수는 접선 기울기가 0인 곳이 곧 최솟값.

# 경사하강법 대 정규방정식

| | 정규방정식 | 경사하강법 |
|---|---|---|
| 반복 | 불필요 — 정확 | 학습률 튜닝, 스텝 수 필요 |
| 계산 비용 | $(X^T X)^{-1}$ — 특징 개수 $n$ 의 **세제곱**에 비례 | $n$ 이 커져도 매 반복 비용은 **선형**으로만 증가 |
| 쓰이는 곳 | 특징이 **적을** 때 | 특징이 **많을** 때 (수천∼수만) |

특징이 적을 땐 정규방정식, 많을 땐 경사하강법을 쓰는 이유다.

# 손으로 한 번: 정규방정식으로 직접 풀어보기 (1/5)

bias 열을 추가한 행렬:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$$

# 손으로 한 번: 정규방정식으로 직접 풀어보기 (2/5)

bias 열을 추가한 행렬:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$$

$X^T X$ 계산:

$$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$$

# 손으로 한 번: 정규방정식으로 직접 풀어보기 (3/5)

$2 \times 2$ 행렬 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 의 역행렬 공식 $\frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ 적용: $ad - bc = 3 \times 14 - 6 \times 6 = 6$

이므로:

$$(X^T X)^{-1} = \frac{1}{6}\begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix}$$

# 손으로 한 번: 정규방정식으로 직접 풀어보기 (4/5)

$2 \times 2$ 행렬 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 의 역행렬 공식 $\frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ 적용: $ad - bc = 3 \times 14 - 6 \times 6 = 6$ 이므로:

$$(X^T X)^{-1} = \frac{1}{6}\begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}\begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} = \begin{bmatrix} 15 \\ 34 \end{bmatrix}$$

# 손으로 한 번: 정규방정식으로 직접 풀어보기 (5/5)

$2 \times 2$ 행렬 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 의 역행렬 공식 $\frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ 적용: $ad - bc = 3 \times 14 - 6 \times 6 = 6$ 이므로:

$$(X^T X)^{-1} = \frac{1}{6}\begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}\begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} = \begin{bmatrix} 15 \\ 34 \end{bmatrix}, \qquad \boldsymbol{w}^* = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix}\begin{bmatrix} 15 \\ 34 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$\boldsymbol{w}^* = [1, 2]$ — **정확히** 데이터를 만들 때 쓴 참값 $y = 2x + 1$ 과 일치. 경사하강법은 "근접"했을 뿐, 정규방정식은 **단 한 번의 행렬 계산으로 정확히** 도달.

# 실습: numpy로 정규방정식 → GD와 비교

```python
import numpy as np

X_raw = np.array([1.0, 2.0, 3.0])
y = np.array([3.0, 5.0, 7.0])
m = len(X_raw)
X = np.column_stack([np.ones(m), X_raw])  # bias 열추가

w_star = np.linalg.inv(X.T @ X) @ X.T @ y
print("정규방정식 해:", w_star)            # [1. 2.]

# 경사하강법으로충분히오래스텝  (2000) 돌리면같은값에
w0, w1, alpha = 0.0, 0.0, 0.1
for _ in range(2000):
    pred = w0 + w1 * X_raw
    err = pred - y
    w0 -= alpha * np.sum(err) / m
    w1 -= alpha * np.sum(err * X_raw) / m
print("GD스텝(2000):", w0, w1)             # 1.0, 2.0
```

# 두 방법은 왜 같은 지점에 도달할까

**핵심**

두 방법이 결국 같은 지점에 도달하는 것은 **우연이 아니다** — $J(\boldsymbol{w})$ 가 **볼록함수**라서 유일한 전역 최솟값을 가지므로, “미분해서 한 번에 푼다”와 “조금씩 내려간다”는 서로 다른 두 길이 **같은 목적지**로 이어지는 것뿐이다.

# 행렬로 다시 쓰기: 닫힌 형태 해

$J(\boldsymbol{w}) = \frac{1}{2m}\sum_i (h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)}) - y^{(i)})^2$ 를 벡터 $y = (y^{(1)}, \ldots, y^{(m)})^T$ 로 쓰면 행렬곱으로 단축:

$$J(\boldsymbol{w}) = \frac{1}{2m}\|X\boldsymbol{w} - y\|^2$$

미분은 두 줄 (연쇄법칙: 힌트는 연습문제 2):

$$\frac{\partial J}{\partial \boldsymbol{w}} = \frac{1}{m} X^T (X\boldsymbol{w} - y)$$

이를 $0$ 으로 놓고 $X^T X\boldsymbol{w} = X^T y$ 로 정리하면 $\boldsymbol{w}^* = (X^T X)^{-1} X^T y$ — **정규방정식(normal equations)**. $(X^T X)$ 가 역행렬 가능한 조건은 다음 "자주 하는 실수"가 다룬다.

# 사영: 왜 이름이 “normal(직각)”인가

- $X\boldsymbol{w}$ 는 어떤 $\boldsymbol{w}$ 를 넣어도 $X$ **의 열들이 펼치는 부분공간 (column space)** 안에만 살아 있다.
- “최소제곱으로 $y$ 에 가장 가까운 $X\boldsymbol{w}$ ” = 이 부분공간 안에서 $y$ 에 가장 가까운 점 = 정확히 $y$ **의 사영(projection)**.
- **최적점에서 오차 벡터가 $X$의 모든 열에 수직**

# 조건수: “늘어난 골짜기”를 재는 숫자

$J$ 가 이차함수이므로 $\nabla J = \frac{1}{m} X^T X (\boldsymbol{w} - \boldsymbol{w}^*)$:

$$\boldsymbol{w}^{(t+1)} - \boldsymbol{w}^* = \left( I - \frac{\alpha}{m} X^T X \right) (\boldsymbol{w}^{(t)} - \boldsymbol{w}^*)$$

- $I - \frac{\alpha}{m} X^T X$ = **오차를 줄여주는 곱셈 행렬**. $X^T X$ 의 고유값 $\lambda_1 \le \lambda_2$ ($2 \times 2$) 이면, 매 스텝 두 방향의 오차가 $(1 - \frac{\alpha \lambda_i}{m})$ 배로 줄어듦.
- 수렴 속도는 **더 큰 고유값 방향** — 등고선이 더 늘어지는 방향을 재고, 이 “늘어짐”의 크기가 **조건수** $\kappa = \lambda_{\text{max}} / \lambda_{\text{min}}$.

# $\kappa \approx 46$ vs $\kappa \approx 2$: 같은 골짜기가 아님

| | $x = [1, 2, 3]$ | “균형 잡힌” 데이터 (100개, $x \sim \mathcal{N}(0, 1)$) |
|---|---|---|
| 조건수 $\kappa$ | $\approx 46.1$ | $\approx 2$ |
| 등고선 | 긴 축:짧은 축 $\approx 46{:}1$ 타원 — **zigzag**하며 천천히 하강 | 거의 원형 — 직선에 가까운 경로 |

## 스케일 10배가 조건수를 ~71배로 만들면

같은 데이터에 특징 스케일 10배 ($x = [10, 20, 30]$, 참값 $y = 0.2x + 1$ — 직선은 **같음**) 로 바꾸면 $\kappa : 46.1 \to 3278.7$ — 약 **71배** 커짐. (bias 열은 스케일링되지 않아 정확히 100배는 아니고, 두 고유값이 스케일에 따라 다르게 늘어나 비율이 바뀜.)

# $\kappa$ 가 걸음 수를 정한다: 매 스텝 오차 감소율

매 스텝 오차 감소율은 $(\kappa - 1)/(\kappa + 1)$:

| | $\kappa = 3278$ | $\kappa = 2$ |
|---|---|---|
| 매 스텝 남는 오차 비율 | $\approx 0.9994$ — 100스텝에도 오차의 약 **60%** 남음 | $= 0.333$ — 100스텝이면 오차가 $1$ |

**정량적 용언**

2.1절 “학습률의 함정”이 스케일에 민감하다는 것의 **정량적 용언**: 특징 스케일이 30배로 다르면, 같은 $\alpha$ 로 수렴하는 데 걸리는 스텝 수가 **수만 배**로 달라질 수 있다.
특징 스케일링이 “편의”가 아니라 **경사하강법의 걸음 수를 바꾸는 핵심 전처리**인 이유. (Week 4에서 다시 만남)

# 특징 스케일링: 골짜기를 "둥근" 형태로

| | **raw 스케일** ($x = 50 \sim 130$) | **표준화** ($(x-\mu)/\sigma$) |
|---|---|---|
| 조건수 | $\approx 99033$ | $\approx 1.0$ |
| 안정 학습률 | $\alpha < 2/\lambda_{\max} \approx 0.00022$ 근처의 $\alpha = 0.0002$ | $\alpha < 2$ 근처의 $\alpha = 0.5$ |
| $J < 10^{-6}$ 에 | **464,595 스텝** | **18 스텝** |

스케일링 한 개가 학습 스텝을 약 **25,800배** 줄였다.

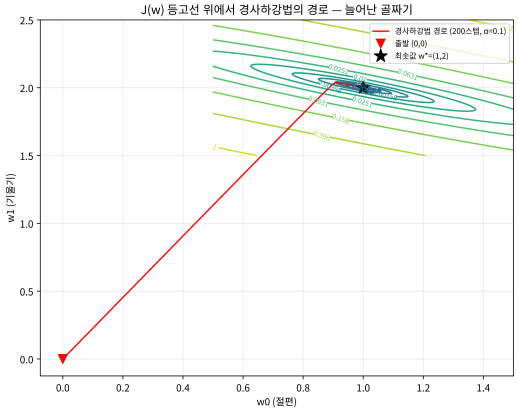
# 스케일링은 답을 바꾸는 게 아니다

- 정규방정식은 두 경우 모두 **한 번의 행렬 계산으로 정확히** $(b = 20, \text{slope} = 1.5)$ 를 낸다.
- 표준화 케이스는 $b = 155$, $\text{slope}_s = 42.43$ 이 나와도, 스케일 변환으로 역변환하면 $\text{slope} = 42.43/\sigma = 1.5$ 가 된다.

**이 절에서 꼭 잡아야 할 핵심**

**스케일링은 답을 바꾸는 게 아니라 경사하강법의 걸음 수를 바꾸는 것.**
정규방정식은 스케일링 여부와 무관하게 같은 답(단, 스케일이 바뀌면 계수 스케일도 같이 바뀜)를 주므로, “답이 바뀌어서” 스케일링을 하는 게 아니다.

# 두 경로, 같은 최솟값




같은 데이터, 같은 출발점에서 정규방정식(직선)과 경사하강법(빨간, 200스텝)이 같은 최솟값 $(1, 2)$ 에 도달.

- 등고선이 $w_0$ **축으로 늘어난** 타원 — 이 데이터셋의 $\kappa \approx 46$ 에 대응.
- 경사하강법은 골짜기 바닥을 일직선으로 타지 못하고 **짧은 축 방향으로 진동**하며 천천히 하강.
- 조건수 46인 골짜기에서 $(\kappa - 1)/(\kappa + 1) \approx 0.958$ — 매 스텝 오차의 95.8%가 남으므로, 200스텝이 “느리게” 보이는 것은 이 수치에서 나온다.

# Gauss-Markov 정리: 최소제곱이 “최선”인 이유

- 최소제곱 해(정규방정식의 해)는 “편하게 풀 수 있는” 해가 아니라 **특정 의미에서 최선** — 가우스(1821)가 먼저 증명, 마코프(1905)가 일반적 형태로 다시 증명해 이름이 합쳐짐.

**Gauss-Markov 정리**

선형모형 $y = X\boldsymbol{w} + \varepsilon$ 에서 오차 $\varepsilon$ 의 기댓값이 0이고 분산이 균일($\sigma^2$ 으로 동일)하며 상호 독립이면, **최소제곱 추정치(OLS, 정규방정식의 해)는 모든 선형 무편차 추정치 중 분산이 가장 작은 것** (BLUE — Best Linear Unbiased Estimator).

# “최선”의 경계와 실전 의미

**Gauss-Markov: “선형 무편차 안에서 최선” — 그 경계를 정확히 말해야 한다.**

# 역사: 가우스가 정규방정식을 발견한 자리

- **가우스의 정규방정식 발견**
- **최소제곱 "손실함수"**와 그를 최적화하는 **"닫힌 형태 해"**가 **같은 시기에, 같은 문제에서 발견**되었다는 것이 중요.

**손실함수 vs 최적화 알고리즘 — 분리될 수 있는 두 작업**

르장드르(1805, 발표)는 최소제곱 손실함수를, 가우스(1809, 『천체 궤도 이론』)는 최소제곱 해의 계산법을 먼저 정식화.
로지스틱회귀(2.3절)와 신경망(Week 9)에서는 이 둘이 **분리된다** — 교차 엔트로피는 이차함수가 아니라서 닫힌 형태 해가 없고, 경사하강법이 독립적으로 발견된 알고리즘이 손실을 최소화.



Carl Friedrich Gauss — 1809년 『천체 궤도 이론』에서 최소제곱 해의 계산법($\boldsymbol{w}^* = (X^T X)^{-1} X^T y$)을 정식화

# 자주 하는 실수: 특징이 겹쳐 행렬이 특이해지면

정규방정식은 $(X^T X)^{-1}$ 이 **존재할 때만** 계산 가능. 두 번째 특징이 첫 번째의 배수(완전 선형종속, 예: “미터 단위 넓이” + “제곱피트 단위 넓이”) 면:

```
X2_raw = np.column_stack([X_raw, 2 * X_raw])   # 두번째  = 첫번째의배  2
X2 = np.column_stack([np.ones(m), X2_raw])
print("det(X^T X) =", np.linalg.det(X2.T @ X2))   # 0.0
np.linalg.inv(X2.T @ X2)   # LinAlgError: Singular matrix
```

행렬식이 정확히 0 — **다중공선성**(multicollinearity) 으로 정보가 $\boldsymbol{w}$ 의 두 성분 중 어느 쪽에 어떻게 배분돼야 할지 수학적으로 결정할 방법이 없어져, **무한히 많은** $(w_1, w_2)$ 조합이 똑같은 예측을 내놓을 수 있게 된다.

# 해결: 무어-펜로즈 준역행렬 (pinv)

inv 대신 유한한 값을 돌려주는 np.linalg.pinv (**무어-펜로즈 준역행렬**, Moore-Penrose pseudoinverse):

```
w_pinv = np.linalg.pinv(X2.T @ X2) @ X2.T @ y
print(w_pinv)                  # [1.  0.4 0.8]
print(X2 @ w_pinv)             # [3. 5. 7.] —정확히들어맞음

w_alt = np.array([1.0, 1.0, 0.5])   # 똑같은예측을내는다른    ** 해
print(X2 @ w_alt)                 # [3. 5. 7.]
print("||w_pinv||^2 =", w_pinv @ w_pinv, "  ||w_alt||^2 =", w_alt @ w_alt)
# 1.8 vs 2.25
```

두 해 모두 세 데이터를 정확히 맞는데, pinv가 고른 해의 $\|\boldsymbol{w}\|^2$ 가 더 작다 — pinv는 무한히 많은 해 중 $\|\boldsymbol{w}\|^2$ **가 가장 작은 해**(최소범위 해, minimum-norm solution)를 골라준다.

# 특이 행렬의 실전 해석과 근종속

- 거의 종속은 조건수 폭발로 $w$ 요동

# 회귀에서 분류로: 같은 도구를 다른 문제에

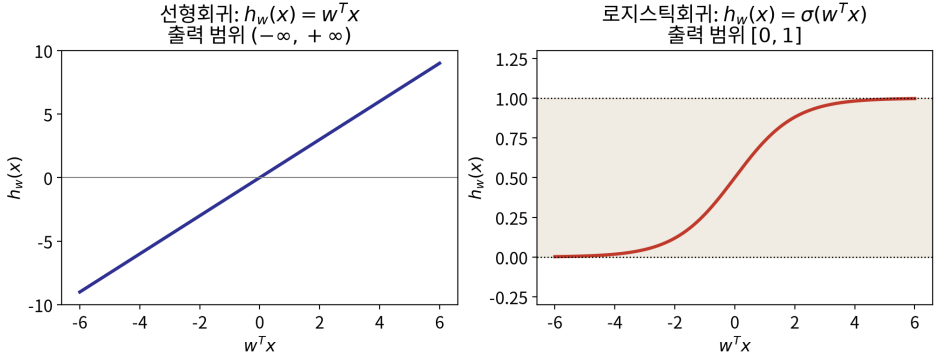
- 로지스틱회귀라는 이름 때문에 헷갈리기 쉽지만, 이건 회귀가 아니라 **분류** 알고리즘.

**핵심 아이디어**

2.1절의 **모델**과 **경사하강법**을 그대로 재사용하고, **"출력을 확률로 바꾸는 변환"**과 **"그 확률에 맞는 새 손실함수"**만 추가하면 된다 — 이번 장 후반부의 핵심.

# 그림으로: 출력 범위가 다르다

같은 $w^T x$ 를 통과시켜도, 출력 범위가 다르다




같은 $\boldsymbol{w}^T \boldsymbol{x}$ 를 그대로 내보내면 범위가 $(-\infty, +\infty)$, 시그모이드를 한 번 통과시키면 범위가 $[0, 1]$ — 로지스틱회귀가 필요한 이유는 바로 이 출력 범위 차이.

# “다리”를 한 눈으로: 비교 표

| 단계 | **선형회귀 (2.1)** | **로지스틱회귀 (2.3)** |
|---|---|---|
| 모델 | $h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x}$ | $h_{\boldsymbol{w}}(\boldsymbol{x}) = \sigma(\boldsymbol{w}^T\boldsymbol{x})$ |
| 출력의 의미 | 연속 숫자 (예: 방 가격) | 확률 (0∼1) |
| 손실함수 | 평균제곱오차 $\frac{1}{2m}\|X\boldsymbol{w} - y\|^2$ | 교차 엔트로피 |
| 손실이 $\boldsymbol{w}$ 에 대해 | **이차함수**(볼록) $\rightarrow$ 정규방정식 존재 | 비이차(볼록) $\rightarrow$ 닫힌 형태 해 없음 |
| 최적화 | 정규방정식 **또는** 경사하강법 | **경사하강법만** |
| 그래디언트 | $\frac{1}{m}X^T(X\boldsymbol{w} - y)$ | $\frac{1}{m}X^T(\sigma(X\boldsymbol{w}) - y)$ — **형태 동일** |

# “그래디언트 형태 동일”이 뜻하는 것

- **시그모이드 미분과 교차 엔트로피 약분**

**“같은 도구를 다른 문제에 쓴다”의 구체적 뜻**

**경사하강법의 코드 줄 자체를 그대로 복사해서, h를 계산하는 부분에 시그모이드 한 줄만 추가하는 것.**
이 패턴 — **모델 구조는 그대로 두고, 출력 해석과 손실함수만 문제에 맞게 바꾼다** — 는 학기 내내 반복.

# 확인 문제: 이 절의 세 가지

1. **왜 둘은 같은 점에 도달할까?** 경사하강법 2000스텝과 정규방정식 1회가 같은 $\boldsymbol{w}^*$ 를 준다. $J(\boldsymbol{w})$ 의 어떤 성질이 "어디서 출발하든 같은 목적지"를 보장하는가? (힌트: 2.1절 FAQ의 볼록함수.)
2. **왜 조건수가 걸음 수를 정할까?** $x = [1, 2, 3]$ 의 $\kappa \approx 46$ 과 $x = [10, 20, 30]$ 의 $\approx 3279$ 를 대입하면 매 스텝 오차 감소율 $(\kappa - 1)/(\kappa + 1)$ 은 각각 $\approx 0.958$ 과 $\approx 0.9994$. $\kappa = 3279$ 일 때 오차가 **절반**으로 줄어드는 데 몇 스텝인가? (힌트: $0.9994^t = 0.5$ — 답 **약 1140스텝**. $\kappa = 46$ 이면 약 16스텝, 약 71배 차이.)
3. **사영과 수직성.** $X^T(X\boldsymbol{w}^* - y) = 0$ 이 "오차 벡터가 $X$ 의 모든 열에 수직"을 뜻하는 이유를 한 문장으로. "normal(직각)" 이름의 출처를 떠올려라.

# 자주 묻는 질문 (1)

**Q. 정규방정식이 있는데 왜 경사하강법을 배우나요?**

- 정규방정식은 선형회귀처럼 $J(\boldsymbol{w})$ 가 이차함수라서 미분=0을 대수적으로 풀 수 있는 **특수한 경우**에만 존재.
- 2.3절 로지스틱회귀부터는 손실(교차 엔트로피)이 $\boldsymbol{w}$ 에 대해 이차함수가 아니라서 **애초에 닫힌 형태 해가 없**.
- Week 9 신경망은 손실이 **비볼록**까지.
- $\rightarrow$ 경사하강법(과 변형들)이 이 학기 전체를 통틀어 실제로 쓰이는 **범용 도구**.
  정규방정식은 "정답과 비교해 경사하강법이 맞게 수렴하는지 검산"하는 용도로 특히 유용.

# 자주 묻는 질문 (2): inv vs solve

**Q. 실전에서 `np.linalg.inv` 를 그냥 쓰면 되나요?**

**권장**

수치적으로는 `np.linalg.solve(X.T @ X, X.T @ y)` 가 역행렬을 명시적으로 구하는 것보다 **더 안정적이고 빠르다** — 역행렬을 구한 뒤 곱하는 것은 **필요 이상의 계산과 오차**를 더한다.
이 절에서는 유도 과정을 눈으로 보여주기 위해 `inv` 를 그대로 썼지만, 실무 코드는 `solve` 습관을.

# 2.3 로지스틱회귀: 시그모이드에서 PR-AUC까지

# 시그모이드: 실수를 확률로

로지스틱함수(시그모이드)는 정확히 "$-\infty \sim +\infty$ 를 $0 \sim 1$ 로 바꾸는" 변환:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $z \to +\infty \Rightarrow \sigma \to 1, \quad z \to -\infty \Rightarrow \sigma \to 0, \quad \sigma(0) = 0.5.$
- $$h_{\boldsymbol{w}}(\boldsymbol{x}) = \sigma(\boldsymbol{w}^T \boldsymbol{x}) = \frac{1}{1 + e^{-\boldsymbol{w}^T \boldsymbol{x}}}$$

  — "$\boldsymbol{x}$ 가 **양성 클래스(1)**일 확률"로 해석.

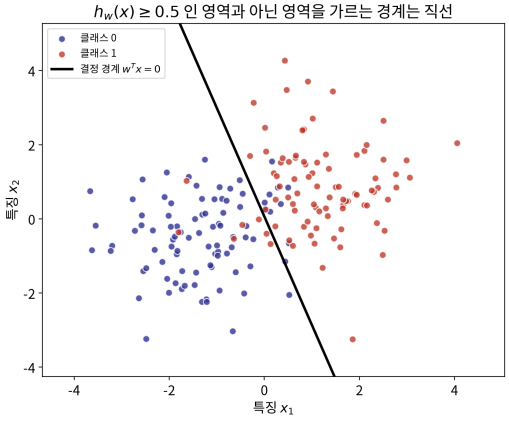
# 결정 경계는 여전히 직선

- 예측은 $h_{\boldsymbol{w}}(\boldsymbol{x}) \geq 0.5$ 면 클래스 1, 아니면 클래스 0.
- **결정 경계는 직선**

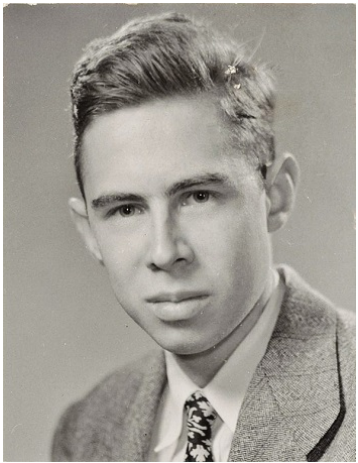
**역사의 한 귀퉁이: 뉴런의 수식 (1943)**

**매컬로크-피츠 뉴런** (McCulloch & Pitts, 1943) — "흥분 입력이 임계값을 넘으면 신호를 보내고, 아니면 보내지 않는 0/1 기계". 뉴런의 실제 출력은 연속적이고 확률적이지만, 이 모델은 수학적으로 다루기 쉬운 첫 뉴런 모델이었다. 그 모델에서 임계값 초과 강도를 "연속적 확률"로 부드럽게 바꿔준 것이 바로 **시그모이드** — 뉴런의 활동전위(activity potential)가 시간에 따라 올라가는 S자 모양을 본떠. (Rosenblatt(1958) 퍼셉트론 이후 활성함수로 정설 → Minsky(1969)의 『Perceptrons』 비판까지 — Week 15에서 다시.)

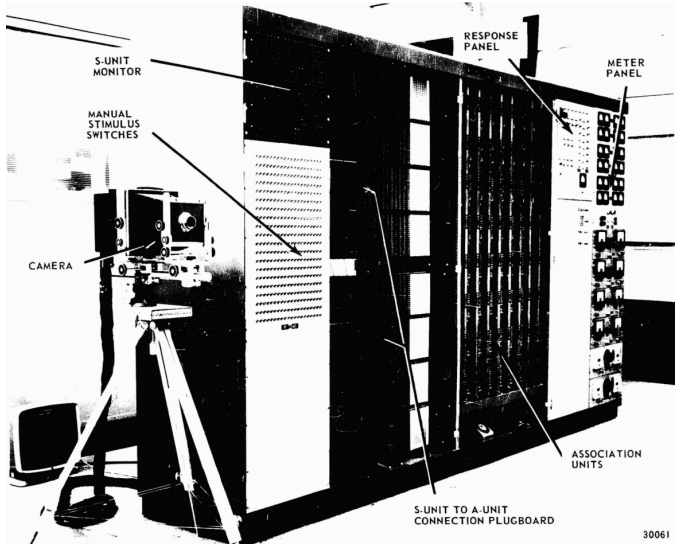
# 그림으로: 결정 경계




예시 데이터에서 $h_{\boldsymbol{w}}(\boldsymbol{x}) = 0.5$ (검은 직선)를 기준으로 한쪽은 클래스 0, 반대쪽은 클래스 1 — 시그모이드는 확률을 만들 뿐, 경계 자체의 **모양**은 $\boldsymbol{w}^T\boldsymbol{x} = 0$ 인 직선(또는 초평면) 그대로다.



Frank Rosenblatt — 코넬 항공연구소, Mark I 퍼셉트론(1958) 개발




Mark I 퍼셉트론 조작 설명서의 실제 장비 — 카메라로 찍은 패턴을 S-unit이 받아 A-unit(연합부)으로 전달

# log-odds 직관: 시그모이드는 “대수적 기울기”

$$z = \log \frac{p}{1-p} = \log \mathrm{odds}(p)$$

- $\mathrm{odds}(p) = p/(1-p)$ = **승산(odds)** — “양성일 가능성 : 음성일 가능성”의 비율.
- **로지스틱회귀는 log-odds를 선형 모델로 설명**

# 계수의 해석: 강도와 방향

- $x_j$ **1 증가 시 log-odds** $w_j$ **변화**

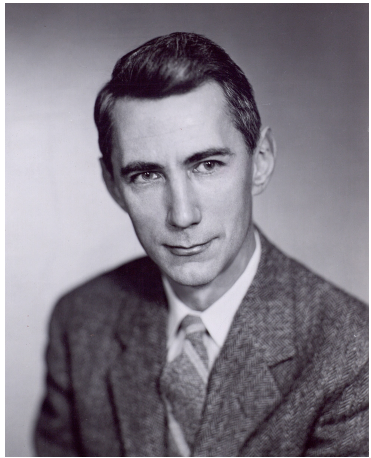
**의학 표준의 이유**

의학적 위험인자 분석에서 로지스틱회귀가 **수십 년간 표준**인 이유.
(Week 7의 결정트리와 비교: 트리 경계는 "$x_1$ 이 5.3 이상"처럼 계수 해석을 주지 않는다.)

# 손실함수: 평균제곱오차로는 안 되는 이유

- **MSE는 non-convex, 지역 최솟값 위험**

**섀넌 정보량:** $-\log_2 p$



Claude Shannon — 1948년 논문 “A Mathematical Theory of Communication”으로 정보이론 창시, 교차 엔트로피 손실의 기원

# 엔트로피, 교차 엔트로피, KL 발산

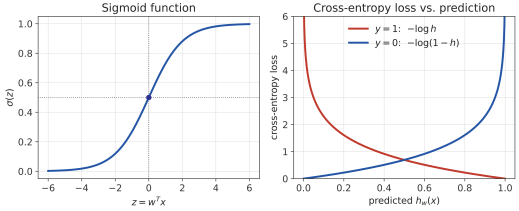
- 엔트로피: 정보량의 기댓값
- 교차 엔트로피: 틀린 분포 $q$ 부호화
- KL 발산: H($p$,$q$) - H($p$)

# 교차 엔트로피를 손실함수로: 직관

- 로지스틱회귀에서 정답 $y^{(i)}$ = “정답 분포” (클래스 1 확률 100% 또는 0%), $h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)})$ = 모델이 예측한 분포.
- **교차 엔트로피 최소화 = 예측 분포를 정답에 가깝게**

**확신을 갖고 틀리면 벌이 커진다**

# 시그모이드와 교차 엔트로피 손실




시그모이드 함수(좌)와 예측 확률에 따른 교차 엔트로피 손실(우) — 정답과 반대로 확신할수록 손실이 무한히 커진다.

- 정답 $y = 1$ 인데 $h_{\boldsymbol{w}}(\boldsymbol{x}) \to 0$ 으로 확신하면 $-\log h_{\boldsymbol{w}}(\boldsymbol{x}) \to \infty$ — 손실이 **무한히** 커짐.
- **확신을 갖고 틀리면 그만큼 크게 벌**.

신기하게도 이 손실을 미분하면 2.1절 선형회귀와 **똑같은 형태**의 그래디언트:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left( h_{\boldsymbol{w}}(\boldsymbol{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

$\to$ 경사하강법 구현은 2.1절과 거의 동일.

# 로지스틱 경사하강법: 시그모이드 한 줄만 추가

```python
import math

def sigmoid(z):
    return 1 / (1 + math.exp(-z))

def logistic_gradient_descent(X, y, alpha, epochs):
    m, n = len(X), len(X[0])
    w = [0.0] * (n + 1)
    for _ in range(epochs):
        grad = [0.0] * (n + 1)
        for i in range(m):
            pred = sigmoid(w[0] + sum(w[j+1] * X[i][j] for j in range(n)))
            error = pred - y[i]
            grad[0] += error
            for j in range(n):
                grad[j+1] += error * X[i][j]
        for j in range(n + 1):
            w[j] -= alpha * grad[j] / m
    return w
```

# 확인 문제 1: 시그모이드의 미분

- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (연습문제 5의 힌트).
- $0 < \sigma(z) < 1$ 에서 항상 **양수**, $\sigma(z) = 0.5$ (즉 $z = 0$) 에서 **최대값 0.25**.
- 실습 노트북 §3: 수치미분 vs 해석미분 비교, 최대 오차 $\approx 2 \times 10^{-4}$ 로 확인.

**직관**

시그모이드는 **"가장 많이 틀린 예측(확률 0.5)에서 가장 민감하게 반응하고, 이미 확신을 가진 예측(0 또는 1에 가까움)에서는 거의 움직이지 않는다"** — 그래서 경사하강법이 **학습 초반에는 빠르게, 후반에는 천천히** 수렴한다.

# 손으로 한 번: 3샘플 로지스틱회귀, 단계 0

한 특징 $x \in \{1, 2, 3\}$, 정답 $y = [1, 0, 1]$, 초깃값 $w_0 = w_1 = 0$, 학습률 $\alpha = 0.5$.

**단계 0**: 모든 예측 $h = \sigma(0) = 0.5$.

$$J = -\frac{1}{3}\left[2\log 0.5 + \log 0.5\right] = -\log 0.5 = 0.6931$$

**단계 1**: 그래디언트 —

$$\frac{\partial J}{\partial w_0} = \frac{1}{3}\sum_i (h_i - y_i) = \frac{1}{3}[(0.5-1) + (0.5-0) + (0.5-1)] = -0.1667$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{3}\sum_i (h_i - y_i)x_i = \frac{1}{3}[(0.5-1)(1) + (0.5-0)(2) + (0.5-1)(3)] = -0.3333$$

# 3샘플 로지스틱회귀: 단계 1 업데이트와 해석

업데이트: $w_0 = 0 - 0.5(-0.1667) = 0.0833$, $w_1 = 0 - 0.5(-0.3333) = 0.1667$.

$h(1) = \sigma(0.0833+0.1667) = 0.5619$, $h(2) = \sigma(0.0833+0.3333) = 0.5381$, $h(3) = \sigma(0.0833+$

손실은 **0.6475** 로 떨어짐.

**해석**

두 샘플 ($x = 1, 3$) 은 양성이므로 모두 $h > 0.5$ 로 "정답 쪽". 유일한 음성 ($x = 2$) 은 가장 작은 기울기로 0.5를 겨우 넘겨 예측.

**"반쯤 맞고 반쯤 틀린 예측"을 받으면 그래디언트도 그만큼 작아진다** — 이것이 직관.

# “정확도”만으로는 안 되는 이유: 정확도의 함정

| | 환자 1000명 중 10명(1%) 실제 암 | 모델: 무조건 “정상” |
|---|---|---|
| 정확도 | $990/1000 =$ **99%** (매우 높아 보임) | |
| Recall | $0/10 =$ **0%** | 실제 암을 **단 한 명도** 못 잡아 |

**정확도의 함정: Precision / Recall / F1**

# 혼동행렬: 네 개의 숫자

**혼동행렬: TP=18, FP=2, FN=7, TN=73**

| 실제 \ 예측 | 양성 예측 | 음성 예측 |
|---|---|---|
| 실제 양성 | **True Positive (TP)** | **False Negative (FN)** |
| 실제 음성 | **False Positive (FP)** | **True Negative (TN)** |

- **Precision: 양성 예측 중 진짜 양성 비율**
- **Recall: 실제 양성 중 잡아낸 비율**
- **F1: P와 R의 조화평균**

# 손으로 한 번: Precision · Recall · F1 계산

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{18}{18 + 2} = \frac{18}{20} = 0.9$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{18}{18 + 7} = \frac{18}{25} = 0.72$$

$$F1 = \frac{2 \times 0.9 \times 0.72}{0.9 + 0.72} = \frac{1.296}{1.62} = 0.8$$

- **Precision 0.9** = “양성이라고 예측한 20명 중 18명이 진짜 양성”.
- **Recall 0.72** = “실제 양성 25명 중 18명만 잡아냈다” (7명 놓침).

# Precision-Recall 트레이드오프

- 임계값(threshold)을 0.5가 아니라 **0.9로 올리면** — Precision **오름** (확신 있을 때만 양성 판정) / Recall **떨어짐** (애매한 양성을 놓침).
- 임계값을 **낮추면** — Recall **오름** / Precision **떨어짐**.

## PR-AUC

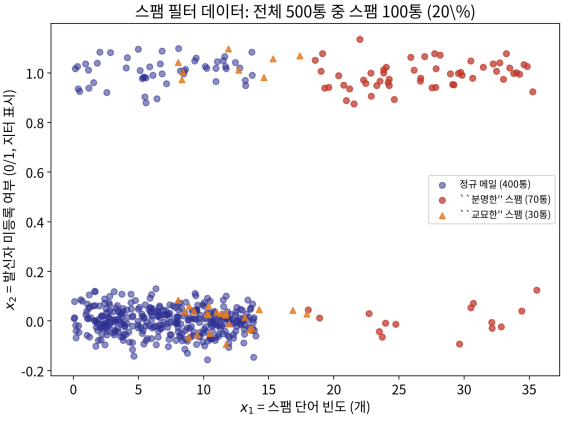
**PR-AUC**(Precision-Recall 곡선 아래 면적)는 이 트레이드오프 **전 구간**에서의 성능을 **임계값 하나에 의존하지 않고** 요약한 지표 — 특히 **클래스 불균형이 심할 때** accuracy보다 훨씬 신뢰할 수 있다.

# 스팸 필터: 임계값을 움직여보자 (설정)

**모델 성능은 임계값 + 모델의 성능**

- **스팸 필터 두 특징**

# 그림으로: 스팸 필터 데이터




"분명한" 스팸(빨강)은 $x_1$(스팸 단어 빈도)이 크고 발신자 미등록 비율도 높아 쉽게 갈리지만, "교묘한" 스팸(주황 세모)은 정규 메일 쪽으로 파고들어 — 임계값을 아무리 옮겨도 완전히 갈라내지 못하는 영역이 남는다.

# 임계값 sweeping: 혼동행렬의 변화

| 임계값 | TP | FP | FN | TN | 정확도 | Precision | Recall |
|---|---|---|---|---|---|---|---|
| 0.05 | 21 | 119 | 2 | 8 | 0.193 | 0.150 | 0.913 |
| 0.10 | 19 | 62 | 4 | 65 | 0.560 | 0.235 | 0.826 |
| 0.20 | 16 | 24 | 7 | 103 | 0.793 | 0.400 | 0.696 |
| 0.30 | 12 | 14 | 11 | 113 | 0.833 | 0.462 | 0.522 |
| 0.40 | 11 | 7 | 12 | 120 | 0.873 | 0.611 | 0.478 |
| 0.50 | 10 | 2 | 13 | 125 | 0.900 | 0.833 | 0.435 |
| 0.60 | 9 | 0 | 14 | 127 | 0.907 | 1.000 | 0.391 |
| 0.80 | 4 | 0 | 19 | 127 | 0.873 | 1.000 | 0.174 |
| 0.95 | 2 | 0 | 21 | 127 | 0.860 | 1.000 | 0.087 |

**Precision/Recall 트레이드오프**

# 비용 최소화 임계값은 0.5가 아니다

- **비용 최소 임계값** $\approx 0.14$
- 임계값은 **비용 함수로 선택**하는 하이퍼파라미터.

## 3. 정확도는 오히려 떨어진다

임계값 0.5에서 정확도 **0.900**, 0.30에서 **0.833** — 0.5보다 **낮다**. 하지만 비용은 132 $\rightarrow$ **124** 로 **떨어진다**.
"정확도가 더 낮은 모델이 더 쓸모있다"는 **역설** — 정확도는 "전체 중 맞힌 비율"을, 비용은 "특정 오류의 가중 합"을 재기 때문.

# 정리: PR-AUC의 기준선은 “무조건 0” 모델

클래스 불균형이 심할 때 PR-AUC의 **하한(baseline)** 은 “**무조건 음성으로 예측하는 모델**”의 정확도 = **양성 비율**.

- 임계값을 **매우 낮추면** “모두 양성” → Recall=1.0, Precision=양성 비율 — PR 곡선의 y-좌표 (Precision)는 최대 양성 비율까지만 올라감.
- 임계값을 **매우 높이면** “모두 음성” → Precision=1.0 (양성을 예측한 게 없으므로), Recall=0 — PR 곡선은 (0,1)에서 시작.

**핵심**

**PR-AUC는 항상 “양성 비율”과 1.0 사이의 값**.
양성 비율 15%면 “무조건 0” 모델도 PR-AUC=0.15를 받는다.

# 검증: 모델 vs “무조건 0” (스팸 데이터)

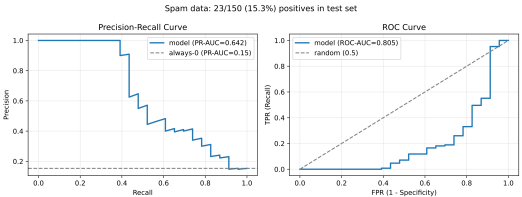
| | 정확도 | PR-AUC / ROC-AUC |
|---|---|---|
| 학습한 모델 | 0.847 ( 0.5) | **0.642** / 0.805 |
| **무조건 0 모델** | 0.847 | **0.153** (= 양성 비율 0.15와 일치) / 0.5 |

**결론**

모델의 정확도(0.847 @ 0.5)와 “무조건 0”의 정확도(0.847)가 **정확히 같은**데도 PR-AUC는 0.642 대 0.153 — **4배 차이**, ROC-AUC도 0.805 대 0.5.

**정확도는 이 두 모델을 구분하지 못하지만, PR-AUC와 ROC-AUC는 구분한다** — 불균형 데이터에서 정확도가 “허위 지표”가 되는 이유.

# PR 곡선과 ROC 곡선




스팸 데이터에서 임계값을 움직일 때의 Precision-Recall 곡선(좌)과 ROC 곡선(우) — “무조건 0” 모델의 PR-AUC는 정확히 양성 비율(0.15), ROC-AUC는 0.5.

- (좌) PR 곡선에서 **점선** = “무조건 0” 모델의 PR-AUC (= 양성 비율 0.15).
- (우) ROC 곡선에서 **점선** = 무작위 모델 (ROC-AUC=0.5).
- 모델 곡선이 점선보다 **위에 있는지** 보기 — 이 차이가 “모델이 양성의 **순서**를 얼마나 잘 짜는지”를 보여준다.

# ROC-AUC vs PR-AUC: 언제 무엇을 보아야 하는가

## ROC-AUC

- TPR-FPR 쌍을 임계값별로 그리는 곡선.
- **클래스 균형이 거의 맞을 때** 좋은 스크리닝 지표.
- 무작위 0.5, 완벽 1.0.
- **불균형이 무너지면** “무조건 0” 모델의 ROC-AUC가 여전히 0.5라, 0.15% 양성에서 ROC-AUC=0.51도 “무작위보다 낫다”로 읽혀 **과대평가**되기 쉬움.

## PR-AUC

- **클래스 불균형이 심할 때** 더 “정직한” 지표 — 기준선이 양성 비율이라, 양성 15%에서 PR-AUC=0.151이면 “무조건 0과 거의 같은 모델”이 바로 읽힘.
- ROC-AUC=0.51은 “무작위보다 약간 낫다”로 읽혀 **절대 척도가 다름**.

## 실무 권장 흐름

**ROC-AUC로 후보 모델 스⊠리닝 → PR-AUC와 비용 곡선으로 최종 임계값 선택.**
ROC-AUC는 “순서 짓기 능력”을 빠르게 비교하고, PR-AUC + 비용 최소화는 “이 문제를 실제로 풀 때 어떤 임계값이 가장 이익”을 결정.

# 자주 하는 실수 (1): 스케일링 없이 경사하강법이 왜곡

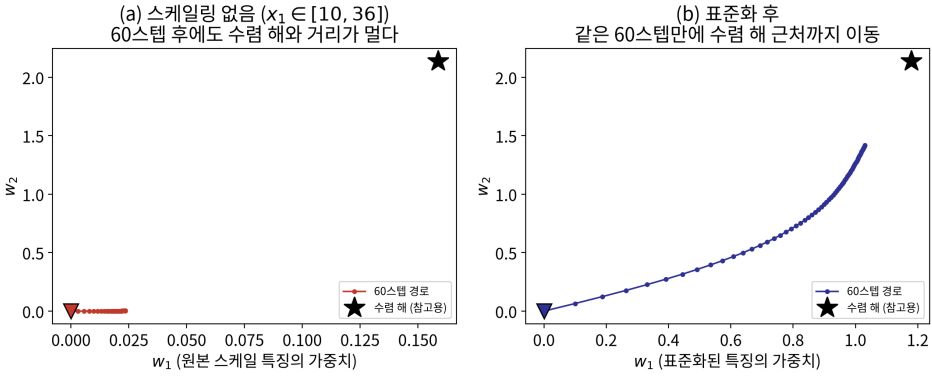
- **시그모이드 그래디언트 스케일 의존성**
- **스케일 왜곡으로 인한 경사하강법 경로 문제**

**실전 표준**

`StandardScaler` 로 스케일링 후 학습 — 이 절의 실습 노트북도 스케일링 후.

# 그림으로: 스케일링이 만드는 경로 차이

같은 60스텝, 같은 데이터 -- 스케일만 다르다




$x_1$(스팸 단어 빈도, 10~36)과 $x_2$(0/1)의 30배 스케일 차이를 그대로 두면 (a) 같은 60스텝을 돌려도 수렴 해 근처에도 못 가지만, 표준화만 하면 (b) 같은 스텝 수로 거의 도달한다.

# 자주 하는 실수 (2): 시그모이드 오버플로

`sigmoid(z)` 를 직접 계산하면 **오버플로**가 날 수 있다.

**실전 해결책 (택 1)**:

- `scipy.special.expit(z)` (C로 구현된, 오버플로 없는 시그모이드)
- **np.where 분산 계산**
- **np.clip 클리핑**

# 자주 하는 실수 (3): 임계값 0.5를 “모델 성능”으로 보기

- **임계값 0.5가 비용 구조에 맞지 않을 수 있다**

## 정확한 순서

“모델이 나쁘다”고 하기 전에:

1. **PR-AUC / ROC-AUC** 로 순위 능력을 **먼저** 확인,
2. **비용 구조에 맞는 임계값**을 sweeping해서 “이 임계값에서 나쁘다”를 확인,
3. 그때 “모델이 나쁘다”고 결론.

순서가 반 거꾸면 “모델을 갈아엎어야 하나”에서 “임계값을 0.14로 바꾸면 된다”로 바뀌지 않는다.

# 실습: sklearn으로 평가 지표 한 번에

```python
from sklearn.metrics import classification_report, average_precision_score

y_true = [1, 1, 1, 0, 0, 0, 1, 0]
y_pred = [1, 0, 1, 0, 1, 0, 1, 0]
y_scores = [0.9, 0.4, 0.8, 0.3, 0.6, 0.2, 0.7, 0.1]  # 임계값 0.5 적용전확률

print(classification_report(y_true, y_pred, target_names=["정상", "스팸"]))
print("PR-AUC:", round(average_precision_score(y_true, y_scores), 3))
```

- classification_report: P/R/F1을 **클래스별 한 번에** 출력.
- average_precision_score: 임계값을 **하나로 고정하지 않고** y_scores(확률) 전체를 보고 PR-AUC 계산 — **순위(ranking) 능력**을 평가.

# 같은 예측에 두 종류의 PR-AUC

| | PR-AUC |
|---|---|
| `y_scores` (임계값 **스윕**) | **0.95** |
| `y_pred` (임계값 0.5 **고정**) | 0.688 |

**핵심**

**같은 데이터, 같은 모델**인데 임계값을 "고정"했느냐 "스윕"했느냐에 따라 지표가 **0.26 차이**.
임계값 하나에 **갱힌** 지표 (임계값 0.5 성능만) 와 모델의 **순서짓기 능력**을 평가하는 지표의 차이가 수치로 보이는 것. (실습 노트북 §6에서 재확인)

# 실습 노트북 지도 + 파이프라인 요약

- **§ 1** 스팸 데이터 만들고 스케일링
- **§ 2** 본문 `logistic_gradient_descent`를 numpy로 학습 (그래디언트 $(h - y) \cdot \boldsymbol{x}$ 확인)
- **§ 3** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 를 수치로 검증
- **§ 4** 임계값 sweeping $\rightarrow$ P/R 트레이드오프와 비용 최소화 임계값 ($\approx 0.14$) 직접 계산
- **§ 5** PR-AUC/ROC-AUC와 "무조건 0" 모델의 기준선 (= 양성 비율) 검증
- **§ 6** 본문 실습 코드 실행

**파이프라인 요약**

선형회귀와 로지스틱회귀는 겉보기엔 다른 문제(숫자 예측 vs 확률 예측)를 풀지만, 뜯어보면 **"선형모델 $\rightarrow$ 손실함수 정의 $\rightarrow$ 경사하강법으로 최소화"**라는 **완전히 같은 파이프라인** — 이 패턴은 **이번 학기 내내 반복**된다.

# 자주 묻는 질문 (1): F1은 왜 조화평균인가

**Q. F1은 Precision과 Recall을 그냥 평균 내지 않고 조화평균을 쓰는 이유는?**

- **산술평균**은 한쪽이 극단적으로 낮아도 다른 쪽이 높으면 값이 크게 낮아지지 않음 — 예: Precision=1.0, Recall=0.01이면 산술평균 0.505로 "꽤 높아 보이지만" 사실상 쓸모없는 모델 (양성을 거의 못 잡아냄).
- **조화평균**은 **작은 값 쪽에 훨씬 민감** — 같은 경우 $F1 = 2 \times 1.0 \times 0.01 / (1.0 + 0.01) \approx$ **0.0198** 으로 훨씬 낮게 나와 "둘 중 하나라도 극단적으로 나쁘면 나쁜 모델"이라는 판단을 더 정확히 반영.

# 자주 묻는 질문 (2): 임계값 0.5는 항상 최선인가

**아니다** — 임계값은 문제의 성격에 맞춰 조정하는 **하이퍼파라미터**.

**암 검진**: 양성 놓침 (FN) 비용 $\gg$ 오진 (FP) 비용
$\rightarrow$ 임계값을 0.5보다 **낮춰서 Recall 올리는 쪽**.

**스팸 필터**: 정상 메일 오거름 (FP) 비용이 더 부담
$\rightarrow$ 임계값을 **높여서 Precision 올리는 쪽**.

**PR-AUC가 유용한 이유**

아직 임계값을 **정하기 전에**, 모델 자체의 **순위 능력**을 먼저 평가할 수 있기 때문.

# 자주 묻는 질문 (3): 불균형에서 무엇을 봐야 하나

**Q. 클래스 불균형이 심한 데이터에서 PR-AUC와 ROC-AUC 중 무엇을 봐야 하나?** $\rightarrow$ **PR-AUC 우선**.

- 양성 15% 데이터: “무조건 0” 모델의 **PR-AUC = 0.15**, **ROC-AUC = 0.5**.
- 학습한 모델의 **PR-AUC = 0.16**이면 “거의 무조건 0과 같은 모델”이 **바로 읽힘**.
- 반면 **ROC-AUC = 0.51**은 “무작위보다 약간 낫다”로 해석되어 **과대평가**되기 쉬움.
- 불균형이 심할수록 **PR-AUC의 기준선(양성 비율)이 더 “정직”**.

ROC-AUC는 스크리닝(후보 모델 비교)에는 여전히 유용하지만, **최종 선택은 PR-AUC + 비용 곡선**으로.

# 자주 묻는 질문 (4): 왜 그래디언트가 같은 형태인가

**Q. 교차 엔트로피의 그래디언트가 선형회귀와 같은 형태 $(h-y)\cdot \boldsymbol{x}$ 인 이유는?**

(1) $\frac{\partial J}{\partial h} = -\frac{y}{h} + \frac{1-y}{1-h}$
(2) $\frac{\partial h}{\partial z} = \sigma(z)(1-\sigma(z)) = h(1-h)$
(3) 세 조각을 곱하면 $h(1-h)$ 가 **통째로 약분**되어 $(h-y)$ 만 남음

## 해석

**"시그모이드의 미분이 정확히 시그모이드의 출입구"**라서, 손실함수의 "벌"과 시그모이드의 "압축"이 **상쇄**되는 것.
이 약분이 없으면 로지스틱회귀의 경사하강법은 선형회귀와 **다른 코드**가 되어야 하는데, 실제로는 **코드 구조가 완전히 동일** — 이 절의 "같은 패턴" 강조의 이유.

# 확인 문제: 4문단

1. $\boldsymbol{w}^T\boldsymbol{x} = -2$ 일 때 $h_{\boldsymbol{w}}(\boldsymbol{x})$ 는 몇이고, 이 예는 음성으로 예측되는가? $\boldsymbol{w}^T\boldsymbol{x} = 3$ 일 때는? (힌트: $\sigma(z)$ 를 대입하고 0.5와 비교.)
2. “무조건 음성으로 예측하는 모델”의 ROC-AUC는 항상 0.5인가? (힌트: 임계값을 움직여 TPR-FPR 쌍을 그려라.)
3. Precision=0.5, Recall=0.9 일 때 F1은 몇인가? (힌트: 조화평균 공식. 단답.)
4. “양성이 5%인 데이터에서 PR-AUC=0.051이면 이 모델은 쓸모가 있다”는 진술은 옳은가? (힌트: PR-AUC의 기준선이 양성 비율임을 떠올려라.)

# 이번 주차 정리

- **2.1 세 부분 뼈대**(모델, 비용함수, 최적화)를 선형회귀에 조립 — 평균제곱오차가 볼록 $\rightarrow$ 지역 최솟값 없음, $\alpha^* = 2/\lambda_{\max}$ 경계로 발산 정확히 계산, diabetes 실데이터로 검증.
- **2.2 정규방정식** $\boldsymbol{w}^* = (X^T X)^{-1} X^T y$ — 미분 0으로 닫힌 해, **사영(수직성)** 해석, 조건수가 걸음 수를 정하고 **스케일링은 걸음 수만 바꾼다** (25,800배 차이), Gauss-Markov("선형 무편차 안에서 최선"), **특이 행렬과 pinv**(최소범위 해).
- **2.3 로지스틱회귀** — 시그모이드를 **log-odds**로 읽는 법, **교차 엔트로피** (Shannon 정보이론), 그래디언트가 $(h - y) \cdot \boldsymbol{x}$ 로 **약분**되어 코드 재사용, "정확도의 함정" $\rightarrow$ 혼동행렬 · P/R/F1 · 임계값 sweeping(비용 최소화 $\approx 0.14$) · **PR-AUC**(기준선 = 양성 비율) · ROC-AUC 스크리닝 $\rightarrow$ PR-AUC로 최종.

## 다음 주 — Week 3. 생성 모델 관점의 분류: 나이브베이즈와 GDA

이번 장의 **판별(discriminative)** 방식(데이터에서 클래스 확률을 직접 추정) 대신, "데이터가 어떻게 만들어졌는지"를 먼저 모델링하고 **베이즈 정리로 뒤집어** 확률을 구하는 **생성(generative)** 방식. 이 관점의 구분이 **학기 전체**에서 분류 방법들을 나눌 기준이 된다.